

Using Wildcard Filtering

In this chapter, you'll learn what wildcards are, how they are used, and how to perform wildcard searches by using the `LIKE` operator for sophisticated filtering of retrieved data.

Using the `LIKE` Operator

All the operators we have studied so far filter against known values. Whether matching one or more values, testing for greater-than or less-than known values, or checking a range of values, the common denominator is that the values used in the filtering are known. But filtering data that way does not always work. For example, how could you search for all products that contained the text *anvil* within the product name? You cannot do that with simple comparison operators; it's a job for wildcard searching.

You can use wildcards to create search patterns that can be compared against your data. For example, say that to find all products that contain the words *anvil*, you could construct a wildcard search pattern that finds the text *anvil* anywhere within a product name.

New Term

Wildcard A special character used to match part of a value.

New Term

Search pattern A search condition made up of literal text, wildcard characters, or any combination of the two.

The wildcards themselves are actually characters that have special meanings within SQL `WHERE` clauses, and SQL supports several wildcard types.

To use wildcards in search clauses, you use the `LIKE` operator. `LIKE` instructs MySQL to compare the search pattern that follows by using a wildcard match rather than a straight equality match.

Note

Predicates When is an operator not an operator? When it is a *predicate*. Technically, LIKE is a predicate, not an operator. Be aware of this term in case you run across it in the MySQL documentation.

The Percent Sign (%) Wildcard

The most frequently used wildcard is the percent sign (%). Within a search string, % means “match any number of occurrences of any character.” For example, to find all products that start with the word `jet`, you can issue the following `SELECT` statement:

► Input

```
SELECT prod_id, prod_name
FROM products
WHERE prod_name LIKE 'jet%';
```

► Output

```
+-----+-----+
| prod_id | prod_name |
+-----+-----+
| JP1000  | JetPack 1000 |
| JP2000  | JetPack 2000 |
+-----+-----+
```

► Analysis

This example uses the search pattern `'jet%'`. When this clause is evaluated, any value that starts with `jet` is retrieved. The % tells MySQL to accept any characters after the word `jet`, regardless of how many characters there are.

Note

Case-Sensitivity Depending on how MySQL is configured, searches might be case-sensitive, in which case `'jet%'` would not match `JetPack 1000`.

Wildcards can be used anywhere within the search pattern, and multiple wildcards can be used. The following example uses two wildcards, one at either end of the pattern:

► Input

```
SELECT prod_id, prod_name
FROM products
WHERE prod_name LIKE '%anvil%';
```

► Output

prod_id	prod_name
ANV01	.5 ton anvil
ANV02	1 ton anvil
ANV03	2 ton anvil

► Analysis

The search pattern '%anvil%' means “match any value that contains the text *anvil* anywhere within it, regardless of any characters before or after that text.”

You can also use wildcards in the middle of a search pattern, although doing that is rarely useful. The following example finds all products that begin with an *s* and end with an *e*:

► Input

```
SELECT prod_name
FROM products
WHERE prod_name LIKE 's%e';
```

It is important to note that, in addition to matching one or more characters, % also matches zero characters. % represents zero, one, or more characters at the specified location in the search pattern.

Note

Watch for Trailing Spaces Trailing spaces can interfere with wildcard matching. For example, if any of the anvils were saved with one or more spaces after the word *anvil*, the clause `WHERE prod_name LIKE '%anvil'` would not match them as there would be additional characters after the final *l*. One simple solution to this problem is to always append a final % to the search pattern. A better solution is to trim the spaces by using functions, as discussed in Chapter 11, “Using Data Manipulation Functions.”

Caution

Watch for NULL Although it might seem that the % wildcard matches anything, there is one exception: NULL. Not even the clause `WHERE prod_name LIKE '%'` will match a row with the value NULL as the product name.

The Underscore (_) Wildcard

Another useful wildcard is the underscore (_). The underscore is used just like %, but instead of matching multiple characters, the underscore matches just a single character.

Take a look at this example:

► Input

```
SELECT prod_id, prod_name
FROM products
WHERE prod_name LIKE '_ ton anvil';
```

► Output

prod_id	prod_name
ANV02	1 ton anvil
ANV03	2 ton anvil

► Analysis

The search pattern used in this `WHERE` clause specifies a wildcard followed by literal text. The results shown are the only rows that match the search pattern: The underscore matches 1 in the first row and 2 in the second row. It does not match the `.5 ton anvil` product because the search pattern matches a single character, not two characters. By contrast, the following `SELECT` statement uses the `%` wildcard and returns three matching products:

► Input

```
SELECT prod_id, prod_name
FROM products
WHERE prod_name LIKE '% ton anvil';
```

► Output

prod_id	prod_name
ANV01	.5 ton anvil
ANV02	1 ton anvil
ANV03	2 ton anvil

► Analysis

Unlike `%`, which can match zero characters, `_` always matches exactly one character.

Tips for Using Wildcards

As you can see, MySQL's wildcards are extremely powerful. But that power comes with a price: Wildcard searches typically take far longer to process than any of the search types discussed to this point. Here are some tips to keep in mind when using wildcards:

- Don't overuse wildcards. If another search operator will do, use it instead.
- When you use wildcards, try to not use them at the beginning of a search pattern unless absolutely necessary. Search patterns that begin with wildcards are the slowest to process.
- Pay careful attention to the placement of the wildcard symbols. If they are misplaced, the data returned might not be what you intended.

Despite their drawbacks, wildcards are important and useful search tools, and you will use them frequently.

Summary

In this chapter, you learned what SQL wildcards are and how to use them in your `WHERE` clauses. You also learned that you should use wildcards carefully and only where necessary.

Challenges

1. Write a SQL statement to retrieve the product name (`prod_name`) and description (`prod_desc`) from the `Products` table and return only products that have the word `toy` in the description.
2. Now let's flip things around. Write a SQL statement to retrieve the product name (`prod_name`) and description (`prod_desc`) from the `Products` table and return only products that don't have the word `toy` in the description. This time, sort the results by product name.
3. Write a SQL statement to retrieve the product name (`prod_name`) and description (`prod_desc`) from the `Products` table and return only products that have both the words `toy` and `carrots` in the description. There are a couple of ways to do this, but for this challenge, use `AND` and two `LIKE` comparisons.
4. This one is a little trickier. I didn't show you this syntax specifically, but see whether you can figure it out anyway, based on what you have learned thus far. Write a SQL statement to retrieve the product name (`prod_name`) and description (`prod_desc`) from the `Products` table and return only products where both the words `toy` and `carrots` appear in the description, in that order (that is, the word `toy` before the word `carrots`). Here's a hint: You'll only need one `LIKE` with three `%` symbols to do this.

This page intentionally left blank